DETECTRA
Bot Review and Coordinated Campaign Detection

Course	DA5402W — Machine Learning Operations
Project	Bot Review Campaign Detection
Repository	github.com/njchathura-boop/Bot-Review-Campaign
Evaluation Branch	main
Team ID	28
Team Members	Chathura N J (DA25M558), Likitha (DA25M584), Karthik(DA25M572)

1. Problem Statement and Objectives

Online marketplaces rely heavily on reviews for product discovery and customer trust, yet review manipulation can occur both as isolated deceptive reviews and as coordinated campaigns. A text classifier can identify suspicious linguistic patterns in a single review, but it cannot, by itself, establish coordination across users, products, ratings, and time. Detectra is a platform for ecommerce review intelligence. Detectra therefore models two distinct decisions.

- Review risk: Estimate whether one review should be inspected by a moderator based primarily on its text.
- Campaign risk: Identify groups of reviews that exhibit coordinated semantic, behavioral, temporal, account, product, and rating patterns.
- Human-in-the-loop governance: Model outputs provide evidence for moderation rather than automatic proof of bot activity.

The Project objective is not only to train accurate models, but to make the complete lifecycle reproducible, testable, observable, deployable, and traceable from source data to a serving prediction.

2. System Architecture

The platform is organized into an offline training path, an online streaming inference path, a serving layer, and an operations/delivery layer. Kafka topics act as asynchronous boundaries in the live path; Airflow schedules bounded ETL and training workflows; Ray executes distributed training; MLflow records experiments and model versions; FastAPI exposes review and campaign APIs; monitoring and CI/CD span the runtime.

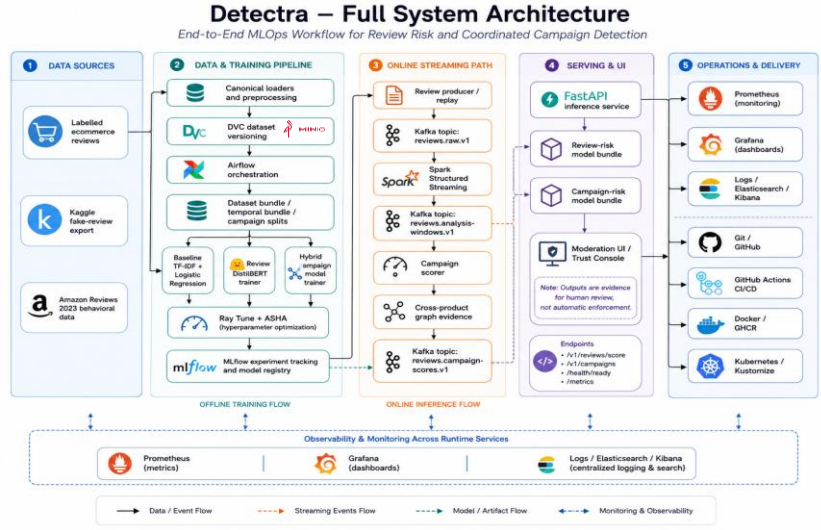


Figure 1. Detectra full-system architecture showing data sources, offline training, online campaign inference, serving, monitoring, and delivery.

3. Data Preprocessing, Versioning, and DVC Storage

3.1. Data Preprocessing

The raw review data was first passed through a validation and canonicalization pipeline before being used for model development. Labelled ecommerce reviews were used for individual review-risk classification, while Amazon Reviews 2023 was retained as an unlabelled behavioral source for campaign modeling. Amazon records preserved their observed user, product, rating, verification, helpfulness, category, and timestamp information; no artificial campaign labels were assigned to the original Amazon observations.

During preprocessing, heterogeneous source fields were converted into a common internal representation. Timestamps expressed in different formats were standardized to UTC, identifiers and ratings were normalized, text formatting inconsistencies were removed, and source-specific label conventions were converted into a consistent binary representation for the supervised review dataset. Invalid records and duplicate review identifiers were rejected rather than silently retained. From 816,216 raw Amazon records, 2,834 invalid records and 1,088 duplicate review IDs were removed, resulting in 812,294 valid behavioral events. For the individual review model, normalized review text was fingerprinted before splitting. Reviews belonging to the same normalized text family were kept within a single partition, preventing duplicate or equivalent text from appearing

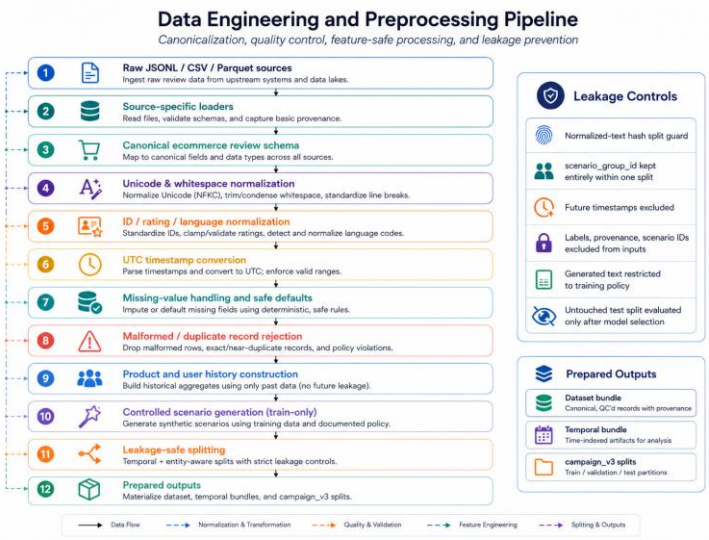


Figure 2. Data engineering and preprocessing pipeline

simultaneously in training and evaluation data. Synthetic text augmentation, where used, was introduced only after the real-data split and was restricted to training data.

For campaign detection, valid Amazon observations were ordered chronologically and enriched with past-only temporal and behavioral information. This included review hour, weekday/weekend behavior, time since product launch, inter-arrival time between reviews, and recent product and user activity. Historical attributes were calculated only from observations occurring before the current event, preventing future information from entering the model.

The processed Amazon activity was then used to learn category-level and global behavioral distributions for posting time, rating behavior, verification behavior, temporal spacing, and observation ranges. These distributions formed the realistic behavioral reference for the controlled campaign scenarios. Generated campaign examples were organized into complete scenario groups, and each group was kept entirely within one data partition to avoid campaign-level leakage.

3.2. Synthetic and Controlled Campaign Data Generation

The Detectra project uses synthetic data for two distinct purposes. First, optional text augmentation is applied only to the training partition of the individual review-risk dataset. Second, a more extensive controlled temporal-scenario generator is used to construct labelled review groups for coordinated campaign detection. These two procedures are intentionally separated because they solve different data limitations.

For individual review classification, real labelled ecommerce reviews remain the primary source of supervision. Synthetic variants are generated only from the real training partition, while the real validation and test partitions remain isolated from generated text. For campaign detection, however, Amazon Reviews 2023 provides realistic behavioral information but does not provide verified labels indicating coordinated manipulation. Consequently, Detectra does not assign fake campaign labels to observed Amazon reviews. Instead, Amazon activity is used as a statistical behavioral reference from which controlled benign and campaign scenarios are generated.

3.3. Behavioral Profile Construction

Before scenario generation, Amazon review events are transformed into a temporal behavioral dataset. The events are ordered chronologically and enriched using past-only features, including review hour, weekday, time since product launch, time since the previous product review, time since the previous user review, and recent user/product activity counts. The implementation deliberately computes these quantities using only records available before the current event, preventing future information from leaking into the generated training features.

From these observed events, we learn category-level and global distributions for:

- UTC review-hour distribution;
- rating distribution;
- verified-purchase distribution;
- temporal/inter-arrival behavior;
- product/category observation periods.

The implementation converts observed counts into a probability distribution, summarizes numerical timing variables through quantiles, and later samples from these learned distributions using weighted choice. Therefore, generated behavioral attributes are anchored to empirical Amazon patterns rather than being independently sampled from arbitrary uniform distributions.

Conceptually,

$$X^{organic} = P_c \text{ where } P_c \text{ denotes a product category}$$

The purpose is to give the model examples of normal combinations of:

$$Time + rating + verification + activity\ level + text.$$

These observations are assigned

$$Y_{campaign}=0$$

3.4. Controlled Scenario Families

Generated observations are distributed across two benign/control behaviors and six campaign behaviors. The project documentation identifies the scenario families as organic traffic, legitimate launch bursts, coordinated positive attacks, coordinated negative attacks, paraphrased campaigns, unusual-hour bursts, slow-drip activity, and campaigns spanning multiple products.

Synthetic dataset/scenario	Type	Target	What is generated/modified	Generation technique and Why?
Organic Activity	Benign campaign control	0	Normal review groups with realistic products, users, ratings, verification status, text, and timestamps	Empirical profile-conditioned sampling. learns category/global Amazon distributions; samples discrete attributes such as rating/verification behavior are scored weightedly. Generated timestamps remain inside the observed product/category time window.
Legitimate Launch Burst	Hard-negative control	0	Dense group of reviews around the product launch period but without campaign membership	Benign temporal-burst simulation conditioned on launch phase. Product launch time is taken from the catalogue where available or an explicitly labelled earliest-review proxy; generated events are concentrated around launch while remaining negative examples. The model is explicitly evaluated for false positives on legitimate launch bursts.

Coordinated Positive Attack	Campaign	1	Group of reviews exhibiting coordinated favorable rating/text behavior	Scenario-conditioned group simulation with positive-polarity coordination. Built complete groups, and profile-based sampling supplies realistic background metadata; scenario rules impose coordinated positive behavior. The online graph later uses semantic similarity and matching rating polarity as coordination evidence.
Coordinated Negative Attack	Campaign	1	Coordinated unfavorable reviews belonging to the same scenario group	Scenario-conditioned group simulation with negative-polarity coordination. . Same profile-driven generation framework is used, but the scenario imposes coordinated negative-rating behavior instead of positive promotion. This prevents campaign risk from becoming synonymous with only positive-rating manipulation.
Paraphrased Campaign	Campaign	1	Related campaign reviews whose wording differs rather than being exact duplicates	Deterministic lexical augmentation / sentence-family generation. Text augmentation utilities split reviews into sentence components, perform small synonym substitutions, and recombine text; split-specific sentence families prevent generated-text leakage across train/validation/test.
Unusual-Hour Campaign	Campaign	1	Coordinated reviews concentrated at atypical UTC posting times	Temporal distribution-shift simulation. Normal hour distributions are learned from Amazon behavior, and places generated events at selected UTC hours to create a controlled departure from the learned temporal profile.
Slow-Drip Campaign	Campaign	1	Related campaign reviews spread over longer periods rather than appearing in one sharp burst	Inter-arrival-time perturbation / temporally dispersed group simulation. Timing is generated from learned Amazon quantiles, and then scenario behavior produces a deliberately less concentrated temporal pattern while keeping the reviews inside one coordinated scenario_group_id. This forces the model to use more than burst intensity alone.
Cross-Product Campaign	Campaign	1	One coordinated scenario group distributed across multiple product IDs	Multi-entity group assignment. A generated scenario group is assigned across several product IDs rather than one product. At inference, account and semantic graph edges can connect reviews across products/categories, and the resulting component is labelled campaign_scope = cross_product.
Review-Text Augmentation	Individual-review training augmentation	0/1	Variants of existing labelled review text	Deterministic sentence recombination + synonym substitution by extracting reusable sentence units; Also performed small lexical changes and augmentation

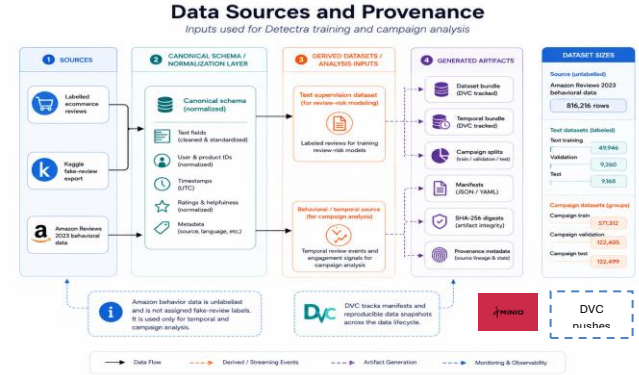
These scenario types were selected to prevent the campaign classifier from learning a large number of reviews mean campaign.

3.5. Data Versioning and Remote Storage

After preprocessing, DVC was used to version the resulting dataset state, while MinIO was used as the remote object store that physically retains the large versioned data objects. This separation allows the project to reproduce a particular training dataset without storing large datasets directly in the source-code repository.

Whenever raw data, preprocessing logic, or relevant parameters change, the preprocessing pipeline is reproduced through DVC. DVC examines the dependencies of each processing stage and computes content hashes for the resulting data. These hashes uniquely represent the dataset state

Figure 3. Data-source and provenance, and architecture separating labelled text supervision from unlabeled behavioral/temporal evidence and DVC-tracked outputs.



produced by that execution. If the contents remain unchanged, the same version is retained; when the data changes, a new version is identified.

Once the processed dataset has been validated, DVC places its content into a local content-addressed cache. The dataset is then pushed from the DVC cache to MinIO through MinIO's S3-compatible object-storage interface. MinIO therefore stores the actual large data objects, while DVC maintains the information required to determine which objects belong to a particular dataset version. This design also supports collaboration and reproducibility. Persistent storage is used for MinIO so that restarting containers or workloads does not remove the remotely stored dataset objects.

Figure 3. The data is cleaned and stored in DVC remote Minio s3

4. Workflow Orchestration and Distributed Training

In Detectra Apache Airflow is used as the outer scheduler for finite ETL and model-training workflows. Rather than performing heavyweight training inside an Airflow worker, the DAG submits work to Ray and waits for job completion. This keeps workflow orchestration separate from distributed compute scheduling. The data-preparation stage validates and processes the source datasets, constructs temporal and behavioral information, and produces reproducible training inputs. Campaign train, validation, and test partitions are then generated while preserving complete campaign groups to prevent leakage. The review-model training stage submits the individual-review DistilBERT training workload to Ray, where Ray Tune performs distributed hyperparameter search.

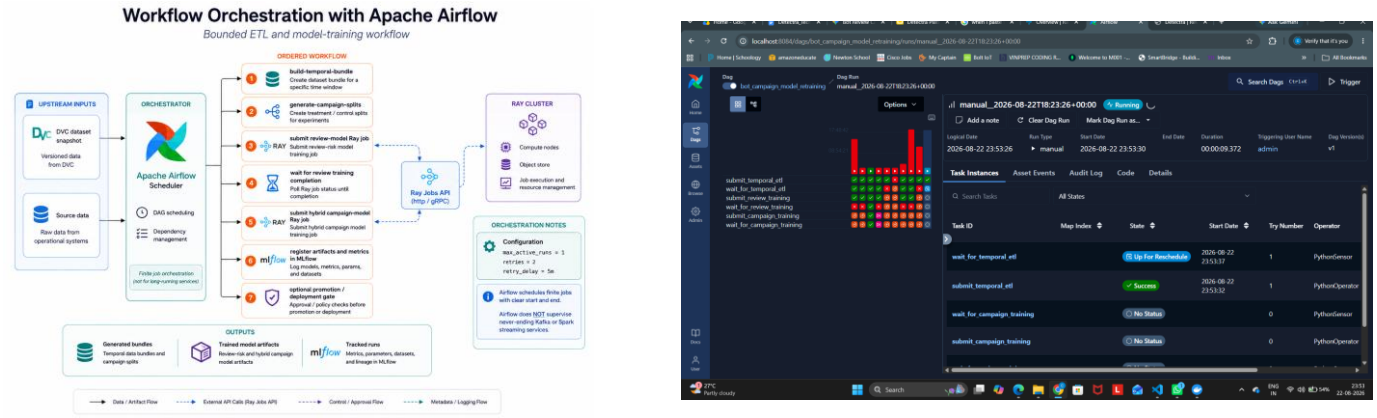


Figure 4-5. Conceptual Airflow orchestration flow for bounded data preparation and Ray-based model-training jobs, with MLflow used for durable experiment tracking.

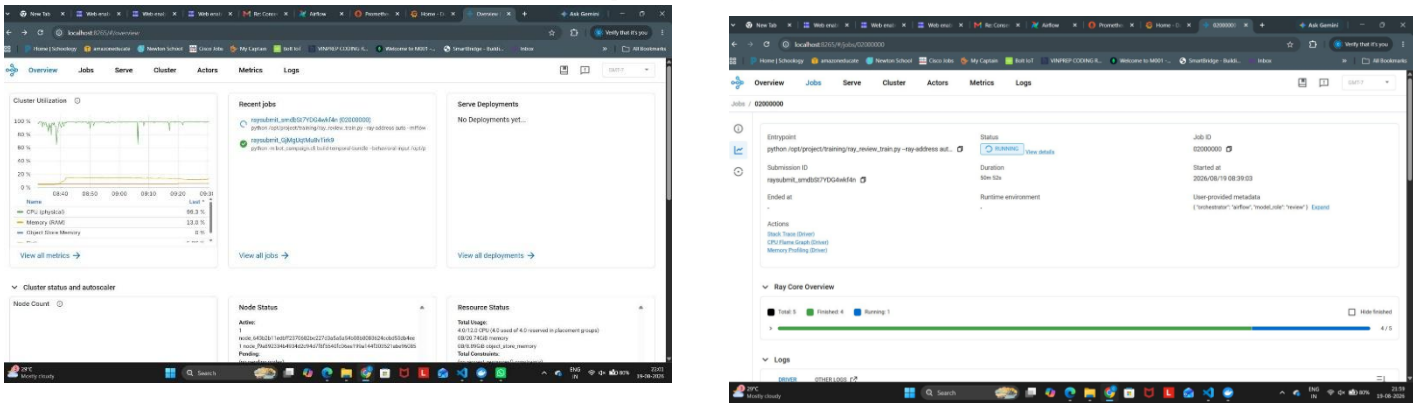
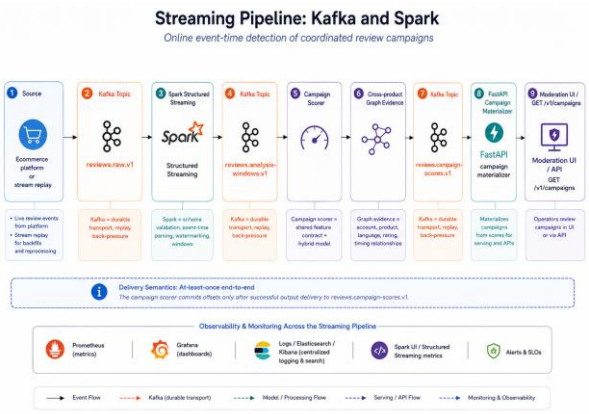


Figure 6. Actual Ray cluster overview showing recent jobs, cluster utilization, node status, and resource status during project execution.

5. Online Streaming Pipeline with Kafka and Spark

To demonstrate coordinated campaign detection under a realistic streaming setting, controlled review events were replayed as individual ecommerce transactions rather than directly supplying a pre-aggregated campaign to the classifier. Kafka was used as the event-transport layer, while Spark Structured Streaming performed event-time validation, routing, and candidate-window construction. During the cross-product attack demonstration, multiple related reviews were injected within a narrow time interval with similar review language and aligned rating polarity while targeting more than one product. A concrete demonstration publishes six coordinated positive reviews across two products, creating controlled temporal, semantic, rating, and cross-product evidence. Each review was published independently to Kafka, meaning that the downstream detector did not receive any explicit indication that the events belonged to the same campaign. Kafka first receives the canonical review events and provides durable asynchronous transport to the streaming pipeline.



The producer validates required fields, adds the expected schema version, uses idempotent publishing with acknowledgements, and verifies successful delivery.

Figure 7. Online Kafka and Spark pipeline for review-event ingestion, event-time windowing, campaign analysis, score publication

Spark Structured Streaming continuously consumes these events, parses their timestamps into UTC event time, applies a two-hour watermark for late-event handling, and constructs overlapping one-hour analysis windows sliding every ten minutes.

Each review is routed through multiple candidate paths based on product identity, account identity, and meaningful normalized text tokens. The semantic routing path is not restricted by product category, allowing related reviews associated with different products to appear in a common candidate window. This is particularly important for detecting campaigns that deliberately distribute activity across several listings. Spark performs only scalable event-time processing and candidate formation; it does not itself classify the reviews as coordinated.

5.1. DistilBERT Review Representation

DistilBERT is used as the semantic encoder for review text. Each review is tokenized and passed through the pretrained DistilBERT transformer to obtain a contextual representation that captures meaning beyond exact keyword overlap. This allows semantically similar reviews with

different wording to still obtain related embeddings. In the individual review-risk model, DistilBERT is fine-tuned directly for binary review classification. In the campaign pipeline, the same type of encoder is used to represent multiple reviews within a candidate group, enabling semantic similarity to contribute to coordination detection. After this an undirected coordination graph is constructed. Graph edges can be produced by account reuse across products, high semantic similarity with consistent rating polarity, or same-product burst behavior involving unverified.

5.2. Hybrid Campaign Model

The campaign model combines DistilBERT-based semantic representations with 15 numerical temporal and behavioral features derived from the complete review group. These features describe characteristics such as review volume, user/product spread, campaign duration, inter-arrival behavior, rating concentration, verification patterns, launch proximity, off-hour/weekend activity, and recent user/product activity. The resulting campaign score and supporting evidence are published to Kafka and subsequently materialized by the FastAPI service for display in the moderation interface. These representations are fused before the final classification layer to estimate the probability that the discovered review group represents coordinated activity. This is important because text alone cannot capture burst timing, account reuse, cross-product reach, or verification patterns, while numerical features alone cannot capture semantic similarity between differently worded reviews. The hybrid architecture therefore combines what the reviews say with how the reviews collectively behave.

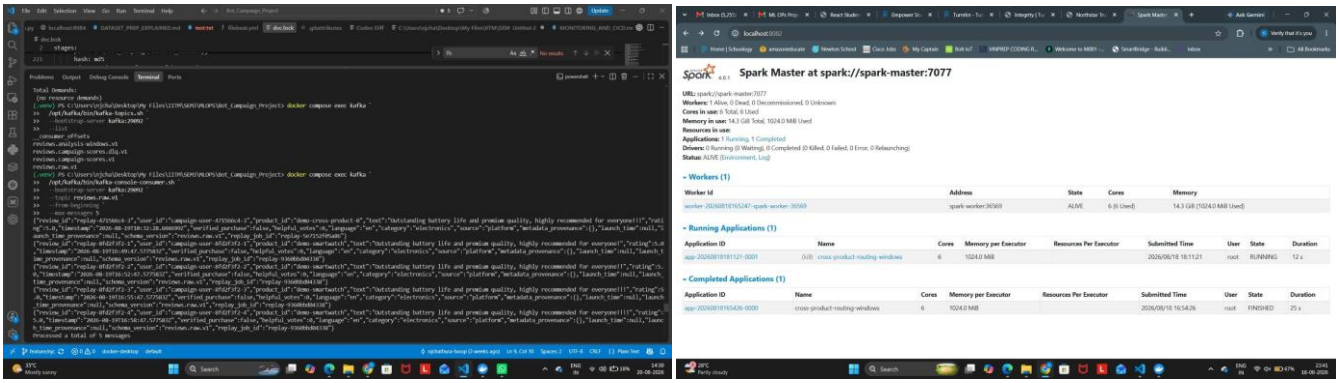


Figure 8. Actual terminal evidence listing Detectra Kafka topics and consuming records from reviews.raw.v1 during a campaign replay.

6. Model Development and Tuning

Two neural models were developed for separate prediction tasks. The individual review-risk model fine-tunes a DistilBERT encoder to transform each review into a contextual semantic representation and produce a binary review-risk score. Ray Tune evaluates alternative hyperparameter configurations, after which validation predictions are used for probability calibration and operating-threshold selection. The corresponding MLflow experiment is `review-risk-distilbert`.

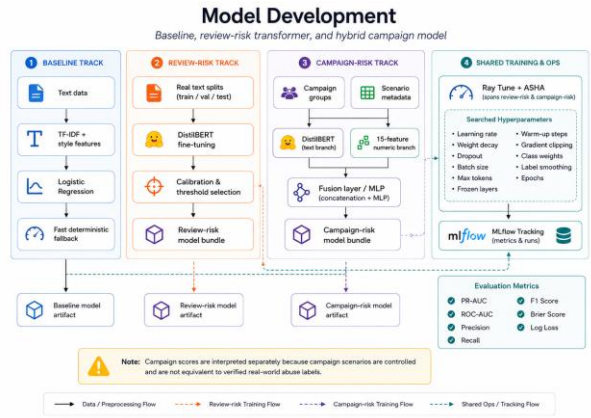


Figure 9. Model-development architecture showing the TF-IDF baseline, review-risk DistilBERT model, hybrid campaign model, Ray Tune with ASHA scheduling, MLflow tracking, and evaluation.

The campaign-risk model is a hybrid neural architecture. A shared DistilBERT encoder creates text embeddings for reviews in a candidate campaign group. Fifteen temporal and behavioural features are processed through a separate numerical neural branch. The text and numerical representations are then fused by a multilayer perceptron classifier. The corresponding MLflow experiment is `bot-campaign-hybrid-distilbert`. Ray Tune evaluated learning rate, weight decay, dropout, batch size, frozen encoder layers, sequence length, review-group size, class weighting, warm-up ratio, gradient clipping, and— for the campaign model—numeric-branch and fusion-layer sizes. The final decision threshold selected for the promoted model was approximately `0.7606`. The final registered model used `distilbert-base-uncased`, a maximum sequence length of 192, dropout of approximately 0.1492, and two training epochs.

6.1 Model Tracks and Hyperparameter Optimization

Three model tracks are maintained: a TF-IDF/style Logistic Regression baseline, a DistilBERT review-risk classifier, and a hybrid campaign model. Ray Tune with ASHA supports hyperparameter optimization, while MLflow records parameters, metrics, artifacts, and selected model versions.

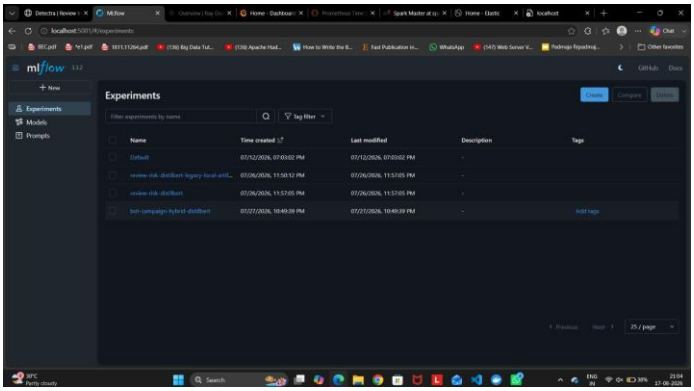
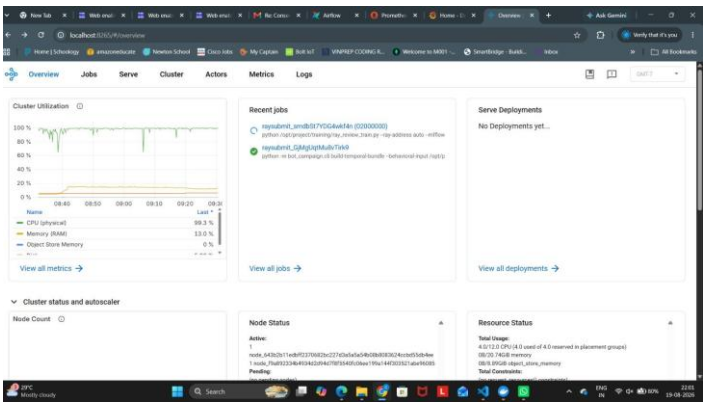


Figure 10. Actual MLflow experiment view showing separate review-risk DistilBERT and bot-campaign hybrid DistilBERT experiments.



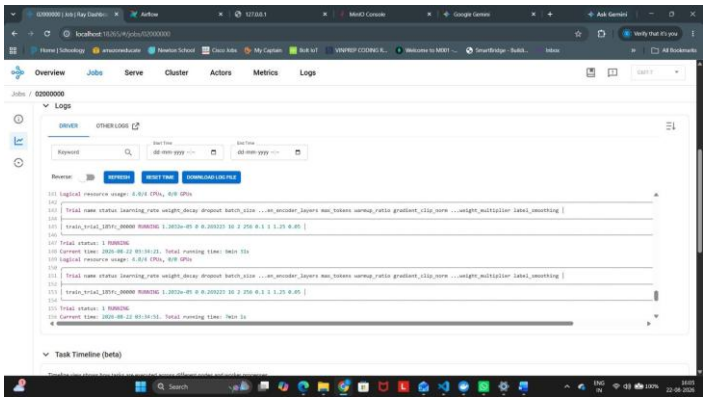


Figure 10. Actual RA Yjob triggered from the AirFlow

7. Monitoring

Detectra implements monitoring at the application, model, streaming, training, infrastructure, and data-quality levels. FastAPI exposes runtime metrics through the /metrics endpoint, which are collected by Prometheus and visualized using Grafana. The monitoring layer tracks request throughput, processed-review volume, p50/p95/p99 API latency, review inference latency, HTTP errors, model availability, review-risk and confidence distributions, campaign alerts, invalid requests, uptime, disk availability, and CPU/GPU/memory utilization. The project provides separate Grafana views for API Operations and Ray Cluster, allowing application behaviour and infrastructure behaviour to be inspected independently. While details like campaign_stream_ candidates_total, campaign_stream_scores_total, campaign_stream_consumer_errors_total and campaign_alerts_total are tracked for the API, CPU usage, memory usage, GPU utilization/memory, worker availability, running trials, failed trial, and resource allocation, training-job status are tracked from the ray endpoint. Human moderation is incorporated into monitoring through counters for confirmed, dismissed, and restored campaign alerts. A rising dismissal rate can indicate an overly aggressive campaign threshold, while increased confirmations may indicate genuinely elevated coordinated-review activity. Therefore, moderator outcomes provide an important operational feedback signal when immediate ground-truth campaign labels are unavailable.

8. Logging

Monitoring tells us what is happening. Logging provides the context needed to understand why it happened. For logging, Detectra uses Filebeat, Elasticsearch, and Kibana in the Kubernetes observability stack. Filebeat reads container logs, enriches them with Kubernetes metadata, and sends them to Elasticsearch, where they are indexed under filebeat-*. Kibana Discover can then filter logs by namespace, container, errors, review-scoring events, or specific review identifiers. This complements metric monitoring: Prometheus and Grafana indicate what changed, while centralized logs help determine why it changed.

Initial alert rules include API p95 latency above 150 ms, API error rate above 1%, excessive Kafka lag, any Spark checkpoint failure, model/version mismatch, sustained GPU-memory saturation, abnormal campaign-alert volume, and abnormal moderator-dismissal rates. Drift alerts require investigation rather than immediate automatic retraining; retraining proceeds only after data and quality review, DVC versioning, leakage-safe dataset creation, Ray/MLflow experimentation, final evaluation, and approval. The FastAPI application exposes operational metrics that Prometheus periodically collects. Important metrics include:

Area	Example metrics	Interpretation
Traffic	api_requests_total, review_scans_total, review_scans_per_second	Amount of application activity
Errors	api_errors_total, api_error_rate, review_invalid_requests_total	Failed API or invalid input requests

Latency	api_request_latency_p50_ms, p95_ms, p99_ms	Normal and tail response latency
Model inference	review_inference_p95_ms	DistilBERT inference performance
Model behaviour	review_mean_risk, review_mean_confidence	Changes in prediction behaviour
Human review	review_needs_human_review_total	Review workload created for moderators
Infrastructure	api_uptime_seconds, api_disk_free_bytes	Runtime and storage health

These metrics allow both performance and behaviour to be monitored. For example, increasing p95 latency with stable request volume may indicate CPU or memory saturation, whereas simultaneous increases in traffic and latency may indicate capacity pressure.
Grafana converts Prometheus time-series data into operational dashboards. Detectra provisions three principal dashboards:

Dashboard	Purpose
Detectra / API Operations	Business/API/model runtime monitoring
Detectra / Ray Cluster	Ray training and compute monitoring

The API Operations dashboard is intended for application and model behaviour, whereas the Ray dashboard focuses on training resources and distributed workers. Prometheus Explore remains available for ad-hoc metric investigation.

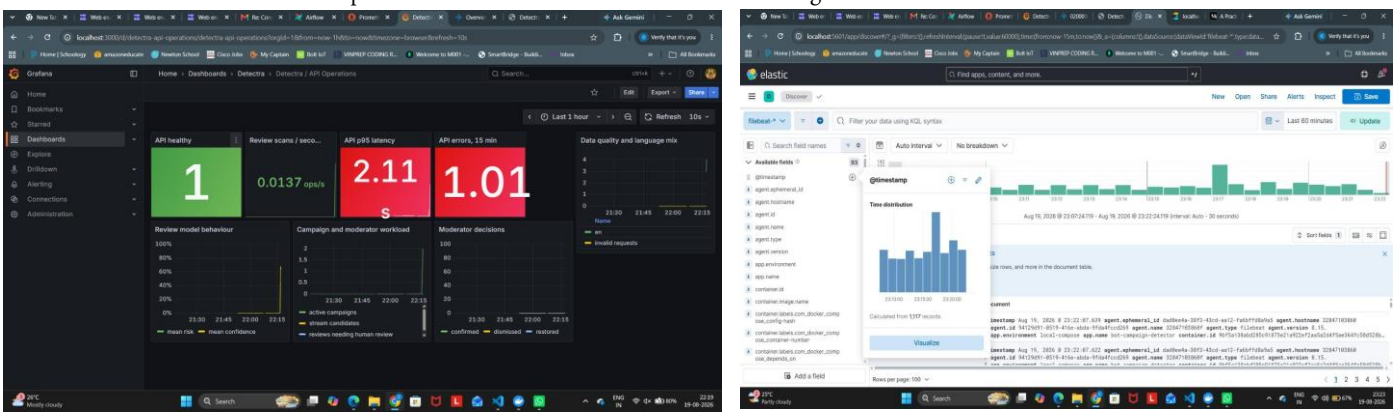


Figure 11. Actual Detectra API Operations Grafana dashboard showing API health, review-scan throughput, p95 latency, API errors, review-model behavior, campaign/moderator workload, and data-quality/language signals and Logging in Kibana

9. CI/CD Pipeline

Detectra uses GitHub Actions as the automation layer that validates source-code changes, builds deployable Docker images, performs security checks, publishes approved images to GitHub Container Registry (GHCR), and deploys release versions to a staging environment.
The project separates this into two workflows:

.github/workflows/ci.yml → Continuous Integration and .github/workflows/cd.yml → Continuous Deployment

The project's CI handles code/model-bundle validation, tests, Docker build and security scanning, while CD performs versioned image publication and staging deployment. GitHub Actions converts these operations into a repeatable pipeline. This also improves reproducibility because a deployed Docker image can be traced back to the exact Git commit from which it was produced. Detectra implements an automated CI/CD pipeline using GitHub Actions through separate ci.yml and cd.yml workflows. Continuous Integration is triggered for pull requests and relevant repository changes and validates the project through dependency installation, promoted-model artifact checks, deterministic smoke training, Airflow DAG compilation, Ruff static analysis, Pytest execution, Docker image construction and Trivy vulnerability scanning. This stage ensures that a change is testable, buildable and acceptable before release. Continuous Deployment is triggered through semantic version tags or manual workflow execution. It verifies the selected model bundles, builds versioned API/UI, training-job and Airflow container

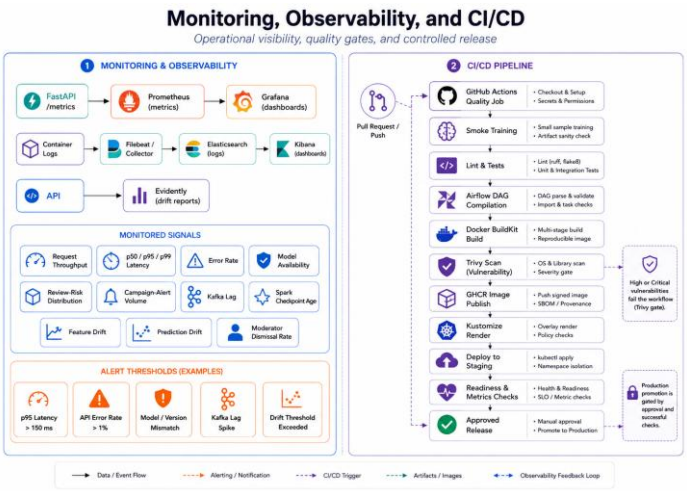


Figure 12. Monitoring, observability, and CI/CD architecture linking FastAPI metrics, Prometheus/Grafana, centralized logging

images, associates them with the Git commit SHA, publishes them to GHCR, performs security scanning, renders deployment configuration using Kustomize and deploys the core stack to staging. Deployment success is verified using workload rollout checks together with FastAPI readiness and Prometheus metrics endpoints. The current automated staging workflow deploys the core API, Ray, MLflow, Prometheus and Grafana stack, while the broader observability and streaming components remain separate deployment profiles. This design provides automated quality assurance, security validation, immutable release artifacts, deployment reproducibility and traceability from the deployed container back to the corresponding Git revision.

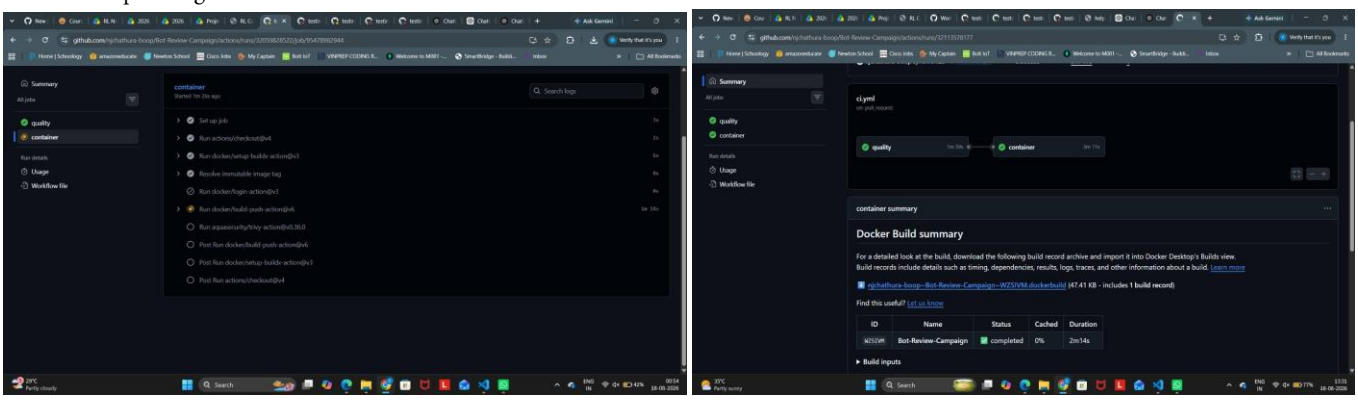


Figure 13-14. Actual completed CI workflow summary showing successful quality and container jobs and a completed Docker BuildKit build record.

10. Serving and Deployment Strategy

10.1 Dockerization

Docker is used to containerize the major Detectra services so that the same software environment can be reproduced across development and deployment. Separate containers are used for components such as the FastAPI application, Ray, MLflow, Prometheus, Grafana, Kafka, Spark, and the supporting observability services. Docker Compose is used for local end-to-end execution, allowing the API, monitoring stack, training services, and Kafka–Spark streaming pipeline to be started as coordinated services with defined ports, volumes, dependencies, and environment variables.

10.2 Kubernetes Deployment

Kubernetes provides a production-style orchestration layer for running the containerized services with controlled resource allocation, health probes, persistent storage, service discovery, and rolling deployment. Kustomize is used to organize different deployment profiles, while GitHub Actions can deploy the core staging stack containing the API, Ray, MLflow, Prometheus, Grafana, and storage. Additional overlays are available for Airflow and the Elasticsearch/Kibana/Filebeat observability stack.

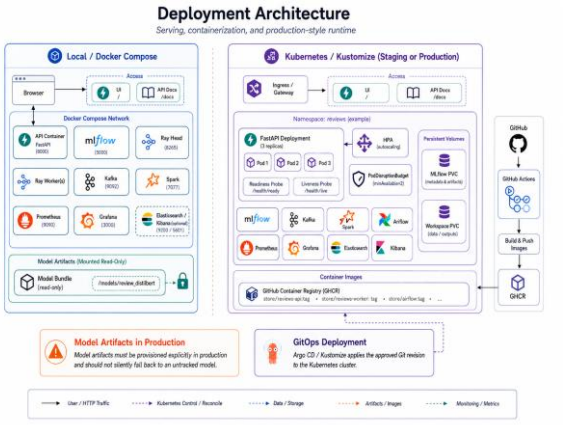


Figure 15. Deployment architecture contrasting the local Docker Compose runtime with the Kubernetes/Kustomize production-style environment.

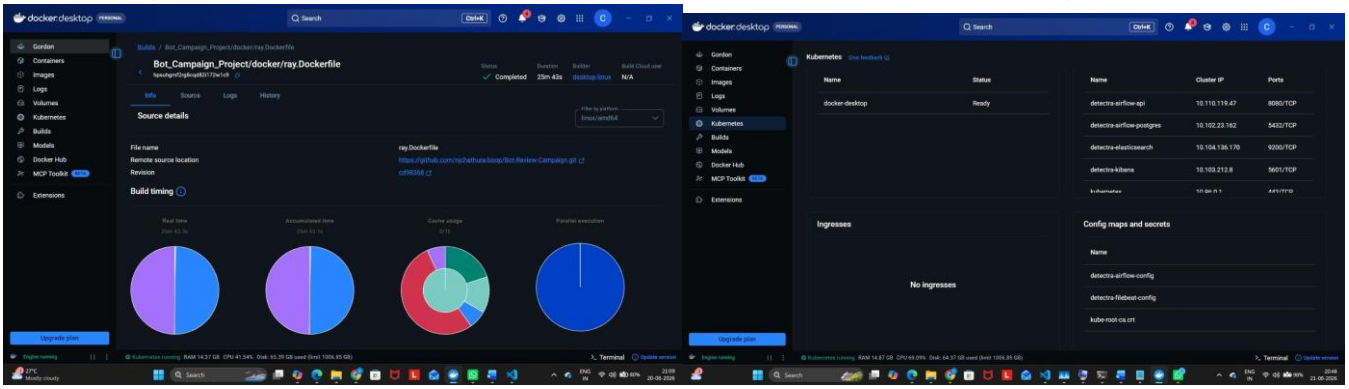


Figure 16-17. Actual Docker Desktop build view showing a completed build of the project Ray Dockerfile and the Docker Desktop Kubernetes view showing the local cluster in Ready state and deployed Detectra services including Airflow, Elasticsearch, and Kibana.

11. Final UI API

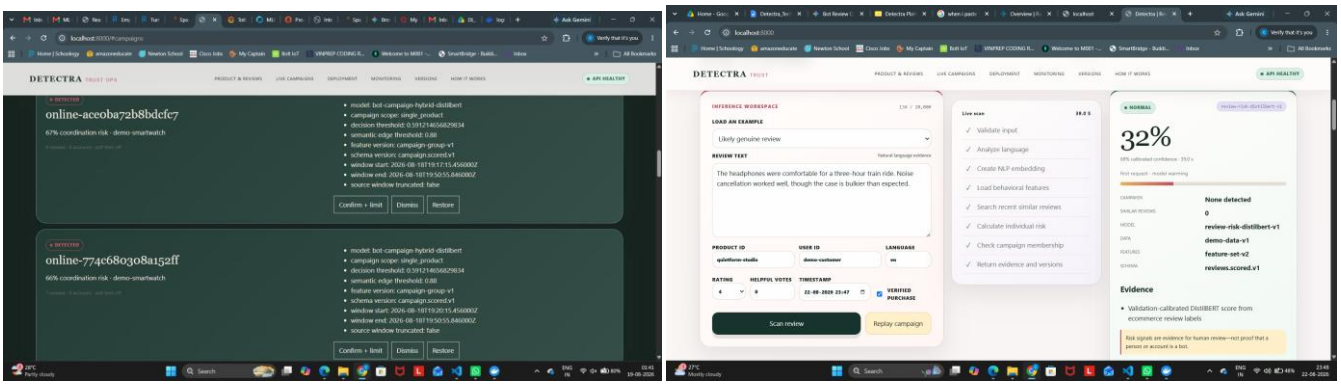


Figure 18 Detectra Review Intelligence API, exposing review scoring, batch scoring, trust, lineage, and campaign endpoints.

12. Results and Discussion

Three models were evaluated in the project:

1. A TF-IDF plus logistic-regression baseline.
2. A DistilBERT review-risk model that scores individual reviews.
3. A hybrid campaign-risk model that combines DistilBERT review embeddings, graph-generated campaign groups, and numerical campaign features.

The baseline was evaluated on a real hold-out review set. The review-risk model was evaluated on the held-out real review test set. The campaign-risk model was evaluated on a controlled synthetic campaign test set.

Model	Evaluation data	PR-AUC	ROC-AUC	F1	Precision	Recall	Brier	Log loss
TF-IDF + Logistic Regression	**9,168 real hold-out reviews**	**0.9016**	**0.8778**	**0.7913**	**0.8356**	**0.7514**	—	—
DistilBERT review-risk model	**Real held-out review test set**	**0.9405**	**0.9264**	**0.8383**	**0.9141**	**0.7742**	**0.1179**	**0.3693**
Hybrid campaign-risk model	**80 controlled synthetic campaign groups**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	—	—

The review model is registered in MLflow under the review-risk-distilbert experiment. The hybrid model obtained perfect scores on the controlled synthetic test groups. These results confirm that the graph grouping, feature generation, and hybrid scoring path work correctly for the generated scenarios.

The review-risk model improved PR-AUC from 0.9016 for the TF-IDF baseline to 0.9405. It also improved ROC-AUC from 0.8778 to 0.9264. This indicates that the DistilBERT model captures linguistic signals that are not fully represented by sparse TF-IDF features. The model's precision of 0.9141 helps reduce unnecessary moderator alerts, while its recall of 0.7742 indicates that some risky reviews may still be missed.

The campaign model operates at a different level. It does not score one review independently. The graph first groups related reviews using user, product, timing, rating-polarity, and semantic-similarity evidence. The hybrid neural model then scores each candidate group using both text embeddings and 15 behavioural/temporal features. Therefore, its metrics should not be compared directly with the individual-review model.

The existing tiny baseline smoke artifact is not treated as a meaningful generalization result. Likewise, controlled campaign scenarios are used to validate pipeline wiring and coordination-feature behavior, not to claim verified real-world accuracy of bot campaigns. This distinction prevents smoke or simulated evidence from being presented as production performance.

13. Challenges, Limitations, and Future Improvements

Key engineering challenges include large review datasets and disk pressure, temporal leakage, memory pressure during transformer trials, model cold-start latency, training-serving consistency, and distinguishing service health from model quality. The implementation addresses these through reproducible data snapshots, past-only features, group-safe splits, controlled Ray concurrency/checkpointing, lazy model loading, shared feature contracts, and separate operational/ML-quality monitoring.

- Collect moderator-confirmed campaign labels and evaluate on a genuine chronological real-world holdout.
- Regenerate the baseline on the same complete real-only test split used for the final review-risk comparison.
- Add multilingual encoders and language-specific calibration.
- Introduce a privacy-reviewed online feature/graph store with freshness monitoring.
- Add shadow-mode canary evaluation, stronger rollback gates, and cost-sensitive thresholding based on moderator workload.

14. Conclusion

Detectra demonstrates an end-to-end MLOps framework for individual review-risk estimation and coordinated campaign detection. The system combines leakage-safe data preparation and DVC/MinIO versioning with Airflow orchestration, Ray-based distributed training, MLflow experiment tracking, Kafka–Spark streaming, FastAPI serving, and Prometheus/Grafana monitoring. GitHub Actions further supports automated testing, container building, security scanning, and release management.

An important outcome of the project was balancing architectural completeness with practical infrastructure limits. Although Kubernetes was designed and tested as a production-style deployment layer, the complete stack placed excessive CPU, memory, and disk pressure on a single laptop. The final implementation therefore uses Docker and Docker Compose as the stable deployment environment, while Kubernetes remains a future scaling option. Overall, the project demonstrates that effective MLOps involves more than model accuracy. It requires reproducibility, experiment traceability, reliable deployment, monitoring, logging, automated delivery, and realistic infrastructure management across the complete machine-learning lifecycle.

15. Course Requirement Compliance Matrix

Course requirement	Detectra implementation / evidence
Git / GitHub	Repository, pull-request workflow, GitHub Actions runs
Apache Airflow	Bounded ETL/model-training orchestration and Ray job submission
Additional data tool	Kafka event transport + Spark Structured Streaming
Data processing	Canonicalization, missing values, cleaning, leakage-safe preprocessing
At least two models	TF-IDF Logistic Regression baseline + DistilBERT; additional hybrid campaign model
MLflow	Experiment tracking and separate review/campaign experiments
DVC	Dataset/pipeline versioning and lock-based reproducibility
FastAPI	Review, batch, trust, lineage, campaign, health and metrics endpoints
Docker	Containerized API/training/services and verified build execution
Monitoring	Prometheus/Grafana metrics + Filebeat/Elasticsearch/Kibana logging
CI/CD	GitHub Actions quality/container jobs, BuildKit, Trivy configuration
Deployment	Docker Compose locally and Kubernetes/Kustomize production-style path
Documentation	Technical report, architecture figures, screenshots, README/runbooks